

Vel Tech Group of Institutions

Name: KISHORE P P
Email: vt27958@veltech.edu.in
Roll no: vt27958
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: M.Tech - Computer Science Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS5L14

VTU_DAA_Week 11_COD

Attempt : 1
Total Mark : 30
Marks Obtained : 30

Section 1 : Coding

1. Problem Statement

Alice needs to assign works to employees in a company. Each employee can be assigned at most one work, and each work can be assigned to at most one employee. Her goal is to maximize the number of employees who are assigned works, ensuring that each work is assigned to exactly one employee and each employee is assigned at most one work.

Help Alice to complete the task.

Input Format

The first line consists of an integer N, representing the number of connections between employees and works.

The next N lines consists of two space-separated integers X and Y, representing a connection where X is the index of an employee (0-based indexing), and Y is

the index of a work (0-based indexing), indicating that employee X can perform work Y.

Output Format

The first line of the output displays "Maximum number of employees that could get works are: a", where a is an integer representing the maximum number of employees that could be assigned works according to the given conditions.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 9

0 0
0 1
0 2
1 0
2 1
3 4
4 3
4 4
5 5

Output: Maximum number of employees that could get works are: 6

Answer

```
#include <stdio.h>
#include <string.h>
```

```
int adj[20][20];
int match[20];
int vis[20];
int n_edges;
```

```
int can_match(int u, int max_id) {
    for (int v = 0; v <= max_id; v++) {
        if (adj[u][v] && !vis[v]) {
            vis[v] = 1;
            if (match[v] < 0 || can_match(match[v], max_id)) {
                match[v] = u;
                return 1;
            }
        }
    }
    return 0;
}
```

```

    }
    }
    }
    return 0;
}

int main() {
    if (scanf("%d", &n_edges) != 1) return 0;

    int max_id = 0;
    for (int i = 0; i < 20; i++) {
        for (int j = 0; j < 20; j++) {
            adj[i][j] = 0;
        }
    }

    for (int i = 0; i < n_edges; i++) {
        int u, v;
        if (scanf("%d %d", &u, &v) == 2) {
            adj[u][v] = 1;
            if (u > max_id) max_id = u;
            if (v > max_id) max_id = v;
        }
    }

    int result = 0;
    for (int i = 0; i < 20; i++) match[i] = -1;

    for (int u = 0; u <= max_id; u++) {
        for (int i = 0; i < 20; i++) vis[i] = 0;
        if (can_match(u, max_id)) {
            result++;
        }
    }

    printf("Maximum number of employees that could get works are: %d\n",
result);

    return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Given an undirected bipartite graph with X vertices on the left side and Y vertices on the right side, the graph may contain multiple edges between the same pair of vertices.

Your task is to determine the minimum vertex cover for this graph.

A vertex cover of a graph is a set of vertices such that every edge in the graph is incident to at least one vertex in the set.

Input Format

The first line contains two space-separated integers X and Y , representing the number of vertices in the left set and right set, respectively.

The second line contains an integer e , representing the number of edges in the bipartite graph.

The next e lines each contain two space-separated integers u and v , representing an edge between: vertex u in the left set and vertex v in the right set

Output Format

The output prints the vertices in the minimum vertex cover, separated by a space.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4 4

4

0 1

0 2

1 3

2 3

Output: 0 3

Answer

```

#include <stdio.h>
#include <string.h>

int X, Y, E;
int adj[101][101];
int matchY[101], matchX[101];
int vis[101];
int visitedX[101], visitedY[101];

int dfs(int u) {
    for (int v = 0; v < Y; v++) {
        if (adj[u][v] && !vis[v]) {
            vis[v] = 1;
            if (matchY[v] < 0 || dfs(matchY[v])) {
                matchY[v] = u;
                matchX[u] = v;
                return 1;
            }
        }
    }
    return 0;
}

void find_cover(int u) {
    visitedX[u] = 1;
    for (int v = 0; v < Y; v++) {
        if (adj[u][v] && !visitedY[v]) {
            visitedY[v] = 1;
            if (matchY[v] != -1 && !visitedX[matchY[v]]) {
                find_cover(matchY[v]);
            }
        }
    }
}

int main() {
    if (scanf("%d %d", &X, &Y) != 2) return 0;
    if (scanf("%d", &E) != 1) return 0;

    for (int i = 0; i < 101; i++) {
        for (int j = 0; j < 101; j++) adj[i][j] = 0;
        matchY[i] = -1;
    }
}

```

```

    matchX[i] = -1;
    visitedX[i] = 0;
    visitedY[i] = 0;
}

for (int i = 0; i < E; i++) {
    int u, v;
    scanf("%d %d", &u, &v);
    adj[u][v] = 1;
}

for (int i = 0; i < X; i++) {
    memset(vis, 0, sizeof(vis));
    dfs(i);
}

for (int i = 0; i < X; i++) {
    if (matchX[i] == -1) {
        find_cover(i);
    }
}

int first = 1;
for (int i = 0; i < X; i++) {
    if (!visitedX[i]) {
        if (!first) printf(" ");
        printf("%d", i);
        first = 0;
    }
}

for (int i = 0; i < Y; i++) {
    if (visitedY[i]) {
        if (!first) printf(" ");
        printf("%d", i);
        first = 0;
    }
}

printf("\n");

return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

You are given an undirected graph with 'num_vertices' vertices and 'num_edges' edges. Your task is to determine whether the given graph is bipartite or not. A graph is bipartite if it can be divided into two disjoint sets of vertices such that each edge connects a vertex from one set to a vertex from the other set.

Input Format

The first line contains two integers separated by a space 'num_vertices' and 'num_edges'. These represent the number of vertices and edges in the graph, respectively.

The next 'num_edges' lines each contain two integers 'u' and 'v' separated by a space, representing an edge between vertex 'u' and vertex 'v'.

Output Format

The output displays a single line indicating whether the given graph is bipartite or not.

If the graph is bipartite, print "The graph is bipartite."

If the graph is not bipartite, print "The graph is not bipartite."

Sample Test Case

Input: 4 4

0 1

1 2

2 3

3 0

Output: The graph is bipartite.

Answer

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main() {
```

```
    int num_vertices, num_edges;
```

```
if (scanf("%d %d", &num_vertices, &num_edges) != 2) return 0;
```

```
int adj[101][101];  
for (int i = 0; i < 101; i++) {  
    for (int j = 0; j < 101; j++) {  
        adj[i][j] = 0;  
    }  
}
```

```
for (int i = 0; i < num_edges; i++) {  
    int u, v;  
    if (scanf("%d %d", &u, &v) == 2) {  
        adj[u][v] = 1;  
        adj[v][u] = 1;  
    }  
}
```

```
int color[101];  
for (int i = 0; i < num_vertices; i++) {  
    color[i] = -1;  
}
```

```
int is_bipartite = 1;
```

```
for (int i = 0; i < num_vertices; i++) {  
    if (color[i] == -1) {  
        int queue[101];  
        int head = 0, tail = 0;  
  
        color[i] = 1;  
        queue[tail++] = i;
```

```
        while (head < tail) {  
            int u = queue[head++];
```

```
            for (int v = 0; v < num_vertices; v++) {  
                if (adj[u][v]) {  
                    if (color[v] == -1) {  
                        color[v] = 1 - color[u];  
                        queue[tail++] = v;  
                    } else if (color[v] == color[u]) {  
                        is_bipartite = 0;
```



```
        break;
    }
}
if (!is_bipartite) break;
}
if (!is_bipartite) break;
}

if (is_bipartite) {
    printf("The graph is bipartite.\n");
} else {
    printf("The graph is not bipartite.\n");
}

return 0;
}
```

Status : Correct

Marks : 10/10